UNIVERSIDAD DEL ISTMO
Facultad de Ingeniería
Sistemas Informáticos en Tiempo Real

PORTEROS ROBOT

Juego de mesa autónomo inspirado en Hyperlaps Initial

Documento técnico del proyecto integrador

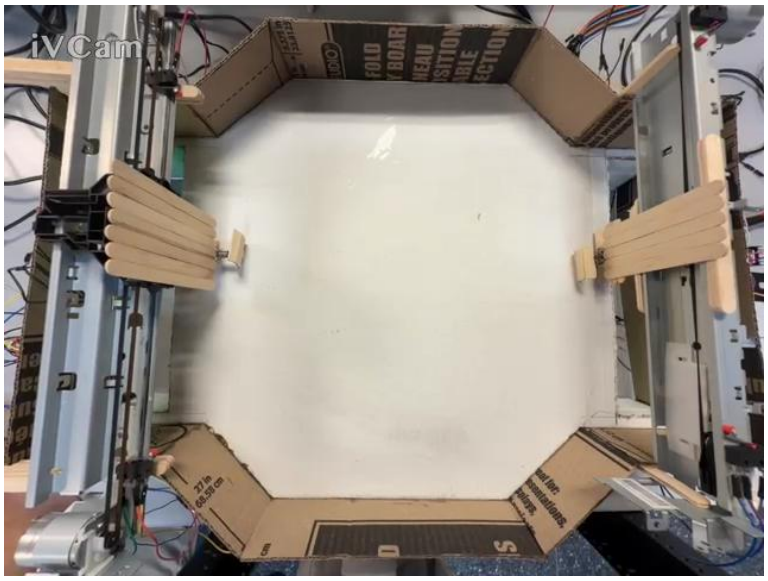


Figura 1. *Vista cenital de la cancha con los dos porteros, tomada por la cámara del sistema.*

Catedrático: Ing. Mario Porras
Integrantes: _____
Guatemala, noviembre de 2026

Contenido

1. Resumen del proyecto 4
2. Requerimientos 5
3. Análisis del problema y planteamiento de la solución 7
3.1 Problema 7
3.2 Alternativas evaluadas 7
3.3 Solución propuesta 7
4. Arquitectura general hardware–software 8
5. Diseño y construcción mecánica 9
5.1 Arena 9
5.2 Naves (porteros) 10
5.3 Mecanismo de golpe 10
6. Diseño eléctrico y electrónico 11
6.1 Alimentación y protección 11
7. Arquitectura de software 13
7.1 Programas y archivos 13
7.2 Ciclo principal de la laptop 13
8. Implementación de tiempo real con FreeRTOS 14
8.1 Justificación de prioridades 15
8.2 Sincronización y comunicación 15
8.3 Condiciones de carrera, bloqueos e inversión de prioridades 15
8.4 Medición de tiempos de respuesta 16
8.5 Uso de memoria 16
9. Estados y operación 17
9.1 Estados del firmware de cada portero 17
9.2 Estados de la partida 18
9.3 Operación 18
10. Percepción, predicción e intercepción 19
10.1 Percepción 19
10.2 Estimación (filtro de Kalman) 20
10.3 Compensación de la latencia 20
10.4 Intercepción 20
10.5 Decisión del golpe 21
10.6 Datos inválidos e incertidumbre 21

10.7 Limitaciones del sensor 21
11. Detección de anotaciones y marcador 23
11.1 Mecanismo 23
11.2 Validación y conteo único 23
11.3 Marcador y ganador 23
12. Plan y resultados de pruebas 24
12.1 Validación del firmware con FreeRTOS 24
13. Registro de problemas, decisiones y cambios de diseño 25
14. Competencias adquiridas 27
15. Evidencia audiovisual 28
16. Limitaciones y trabajo futuro 29
Anexo A. Comandos de operación y prueba 30
Anexo B. Fragmento del firmware: creación de las tareas 30

(Si el índice aparece vacío, en Word: clic derecho sobre él → Actualizar campos.)

1. Resumen del proyecto

Se diseñó, construyó y programó una arena de juego para dos jugadores en la que dos **porteros robot** (las «naves» del enunciado) defienden su portería de forma completamente autónoma. Una cámara cenital observa la cancha; un programa en Python con OpenCV localiza la bola, estima su velocidad con un filtro de Kalman, predice dónde cruzará la línea de cada portero (incluidos los rebotes en las paredes) y envía por puerto serie el objetivo de posición y la orden de golpe a dos Arduino Uno. Cada Arduino ejecuta **FreeRTOS** y controla su carro (motor DC sobre el riel de una impresora Epson L120), protege los extremos con finales de carrera atendidos por interrupción y acciona un solenoide de empuje de 12 V que devuelve la bola.

La partida se gestiona en la laptop: detecta los goles con la misma cámara, lleva el marcador, lo muestra en una pantalla LCD de 16x2 y declara al ganador cuando un jugador llega a 10 goles; luego el marcador vuelve a 0-0 para una nueva partida sin reprogramar nada.

Tabla 1. *Resumen del cumplimiento del alcance mínimo (sección 3 del enunciado).*

Requisito mínimo	Cómo se cumple	Estado
Arena física construida y operativa	Octágono de 400 x 430 mm con dos porterías de 300 mm	Cumplido
Dos porterías y dos naves motorizadas	Carros de impresora Epson L120 con motor DC, banda y finales de carrera	Cumplido
Mecanismo de golpe por jugador	Solenoide push 12 V con bloque de madera, manejado por L298N	Cumplido
Detección automática de la bola	Cámara cenital a 60 fps + visión por color (HSV) + Kalman	Cumplido
Detección automática de anotaciones	Con la cámara: la bola cruza la línea entre postes, rodando hacia la portería	Cumplido
Indicador de 10 puntos por jugador	Marcador en LCD 16x2 y en pantalla; gana el primero en 10 goles (ver 11.3)	Cumplido con variante
Control autónomo de las dos naves	Modo automático: sin intervención humana	Cumplido
Software en Arduino con FreeRTOS	Tareas, colas, mutex, notificaciones, temporizador e ISR (sección 8)	Cumplido (probado en los dos Uno)
Gestión completa de la partida	Inicio, juego, anotación, ganador y reinicio sin reprogramar	Cumplido
Documentación, diagramas, código y videos	Este documento, carpeta del proyecto y registros CSV/MP4	En entrega

2. Requerimientos

Los requerimientos se priorizaron con la escala **Alta** (sin ella no hay partida autónoma), **Media** (mejora el desempeño o la seguridad) y **Baja** (extensión).

Tabla 2. *Requerimientos funcionales.*

ID	Requerimiento	Prioridad	Implementación
RF-01	Localizar la bola en coordenadas de la cancha (mm) en tiempo real	Alta	vision_cancha.py: homografía + HSV
RF-02	Estimar velocidad y dirección de la bola	Alta	Filtro de Kalman (trayectoria.py)
RF-03	Predecir el punto de intercepción con rebotes	Alta	cruce() con reflexión en espejo
RF-04	Mover cada nave al punto calculado	Alta	tControl en el Arduino (T<mm>)
RF-05	Golpear la bola en el momento oportuno	Alta	Criterio de tiempo, distancia y velocidad + K
RF-06	Establecer la posición de referencia de la nave	Alta	Calibración con los dos finales de carrera
RF-07	Impedir choques contra los extremos	Alta	ISR de finales + límites de objetivo 15–205 mm
RF-08	Detectar goles sin contarlos dos veces	Alta	Regla de cruce + velocidad + bloqueo de 2 s y rearme
RF-09	Llevar el marcador, declarar ganador y reiniciar	Alta	Clase Partida + LCD
RF-10	Manejar pérdida de bola y datos inválidos	Alta	Kalman predice 0,15 s; centro tras 1 s; filtros de área y salto
RF-11	Modo manual de prueba	Baja	Teclas a/d/k por el monitor serie
RF-12	Registrar evidencias de cada partida	Media	CSV por foto + video MP4 anotado (tecla R)

Tabla 3. *Requerimientos no funcionales.*

ID	Requerimiento	Prioridad	Valor objetivo / medido
RNF-01	Frecuencia de percepción suficiente	Alta	≥ 30 Hz; medido 64 fps (intervalo p95 = 30 ms)
RNF-02	Latencia total cámara → decisión acotada y compensada	Alta	Medida ≈ 100 ms; se compensa adelantando la posición
RNF-03	Respuesta del control del carro	Alta	Periodo ≤ 16 ms (1 tick); eventos atendidos al instante
RNF-04	Precisión de posicionamiento	Alta	Error típico ±8 mm frente a una paleta de ~100 mm
RNF-05	Seguridad: el solenoide no queda energizado	Alta	Apagado por temporizador del kernel (pulso 350 ms)
RNF-06	Seguridad: frenar si se pierde la comunicación	Alta	Watchdog: 500 ms sin objetivo → freno
RNF-07	Mantenibilidad: mismo firmware en los dos porteros	Media	Autocalibración de sentido del motor y PWM
RNF-08	Recalibración rápida cuando se mueve la cámara	Media	Tecla K (4 clics) + medición de carros

ID	Requerimiento	Prioridad	Valor objetivo / medido
RNF-09	Uso de memoria del Arduino Uno	Alta	Flash 77 %, RAM estática 35 %; pila libre medida ≥ 54 B por tarea

3. Análisis del problema y planteamiento de la solución

3.1 Problema

Una bola rueda por una cancha cerrada a velocidades de 0,3 a 1,5 m/s y rebota en paredes diagonales y rectas. Cada portero recorre 210 mm de riel y tarda entre 0,4 y 0,6 s en cruzarlo, por lo que no basta con seguir la posición actual de la bola: hay que **anticipar** dónde va a cruzar la línea del portero y empezar a moverse antes. Además el golpe debe ocurrir en una ventana corta (decenas de milisegundos) para que el émbolo encuentre a la bola.

3.2 Alternativas evaluadas

Tabla 4. *Decisiones técnicas principales.*

Decisión	Alternativas	Elegida y justificación
Sensado de la bola	Sensor de distancia VL53L0X en cada portero; matriz de sensores; cámara	Cámara cenital: entrega x, y y velocidad de toda la cancha (el VL53L0X solo mide una distancia y no ve los rebotes)
Procesamiento de visión	En el Arduino; en la laptop	Laptop con Python/OpenCV: el Uno no tiene memoria ni CPU para imágenes
Detección de la bola	Resta de fondo; red neuronal YOLO; color HSV	Color HSV: rápido (64 fps), robusto con bola azul sobre fondo blanco; YOLO se probó y era más lento
Mecánica de las naves	Perfil 2020 + GT2 (referencia); riel de impresora	Riel, banda y motor de impresora Epson L120 reutilizados: guiado ya alineado y bajo costo (solución equivalente permitida en 5.1)
Posición del carro	Encoder; posición por tiempo	Dead reckoning por tiempo calibrado contra los dos finales de carrera (no hay encoder disponible)
Golpe	Relé; MOSFET IRFZ44N; puente H L298N	L298N dedicado a los solenoides: el relé disponible estaba dañado; el L298N aísla al Arduino y trae diodos de protección
Detección de goles	Láser/IR en la portería; cámara	Cámara: el sensor IR daba falsos positivos; la cámara ya ve la línea de gol
Sistema operativo	Lazo único (versión 1); FreeRTOS	FreeRTOS: prioridades, eventos por interrupción y comunicación entre tareas (requisito del curso)

3.3 Solución propuesta

El sistema se divide en dos niveles que se comunican por USB serie (Figura 2):

- **Nivel de percepción y decisión (laptop):** cámara, visión, estimación, predicción, decisión de golpe, reglas de la partida y registro de evidencias.
- **Nivel de control en tiempo real (Arduino + FreeRTOS):** posición del carro, control del motor, protección de topes, pulso del solenoide, pantalla y supervisión.

Así la tarea pesada (procesar 64 imágenes por segundo) no compite con el control del motor, y cada Arduino puede reaccionar en microsegundos a un final de carrera aunque la laptop se atrase.

4. Arquitectura general hardware–software

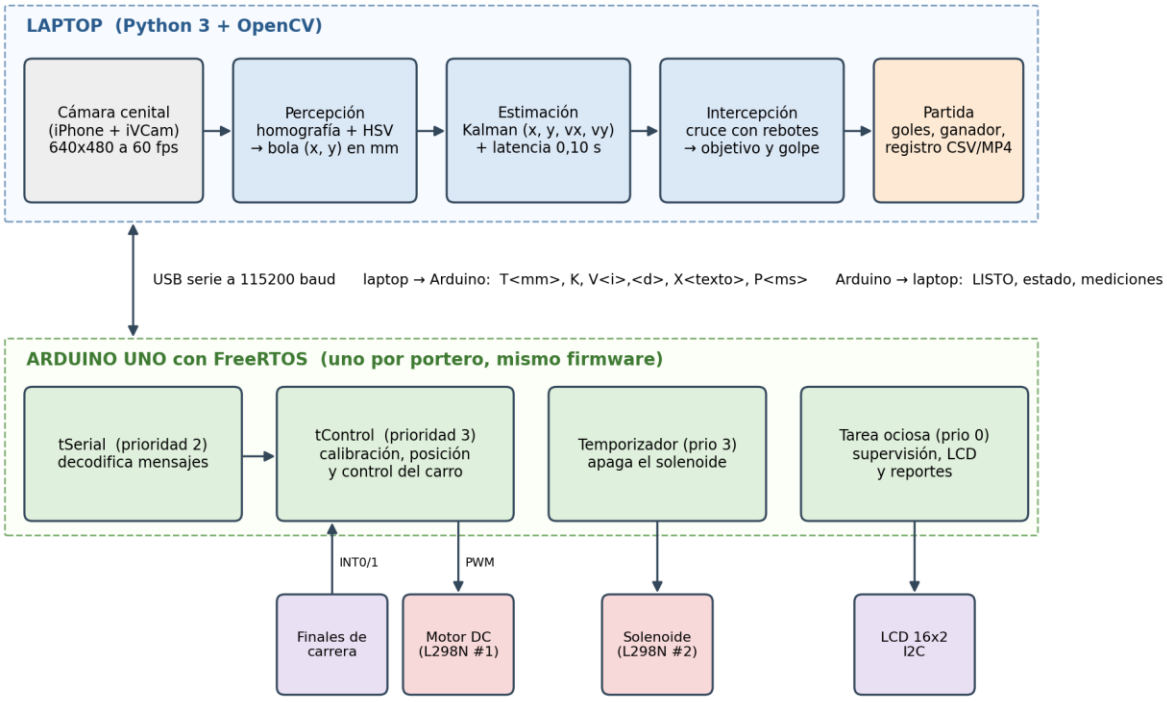


Figura 2. *Arquitectura general: la laptop percibe y decide; cada Arduino controla su portero con FreeRTOS.*

Tabla 5. *Protocolo serie laptop ↔ Arduino (115200 baud, texto terminado en salto de línea).*

Mensaje	Sentido	Significado
T<mm>	laptop → Arduino	Objetivo del carro en mm desde el tope izquierdo (se limita a 15–205)
K	laptop → Arduino	Golpe del solenoide
P<ms>	laptop → Arduino	Duración del golpe (20–600 ms); se envía 350 al conectar
V<i>,<d>	laptop → Arduino	Marcador para la línea 1 de la LCD
X<texto>	laptop → Arduino	Mensaje para la línea 2 de la LCD (máx. 16 caracteres)
c, p, m, a, d, k…	laptop → Arduino	Recalibrar, estado, mediciones de tiempo real y teclas manuales
LISTO	Arduino → laptop	Calibración terminada: el portero puede jugar
Estado / mediciones	Arduino → laptop	Respuestas a p y m, avisos de fallo

5. Diseño y construcción mecánica

5.1 Arena

La arena es un octágono simétrico. Las dimensiones se redujeron respecto a la referencia de 700 x 700 mm porque el recorrido útil de los rieles de impresora disponibles es de 210 mm; con una portería de 300 mm, una cancha más grande dejaría zonas imposibles de defender. Todas las medidas se tomaron con regla y se usan directamente en el software.

Tabla 6. *Dimensiones de la arena construida frente a la referencia.*

Elemento	Referencia del enunciado	Construido	Justificación
Área de juego	700 x 700 mm	400 mm entre caras de los porteros x 430 mm entre paredes rectas	Escalada al recorrido del riel (210 mm)
Geometría	Octágono simétrico	Octágono: paredes rectas de 240 mm arriba y abajo + 4 diagonales	Igual a la referencia
Paredes	379 x 55 x 4 mm, transparentes	Cartón rígido, 75 mm de alto	Material disponible; altura suficiente para la bola
Porterías	Una por lado	Dos, de 300 mm de poste a poste	Mínimo obligatorio de dos jugadores
Bola	Acero pulido	Bola azul	Color con alto contraste para la visión por cámara
Naves	Dos como mínimo	Dos carros de impresora Epson L120	Desplazamiento horizontal sobre riel



Figura 3. *Vista enderezada por homografía: cada píxel equivale a 1/1,5 mm; las paredes rectas quedan en y = 0 y y = 430 mm.*

5.2 Naves (porteros)

Cada portero reutiliza el mecanismo de carro de una impresora Epson L120: riel guía, banda, poleas y motor DC. Sobre el carro se montó la paleta defensora (palitos de madera) y el solenoide. Dos finales de carrera normalmente abiertos marcan los extremos y sirven de referencia de posición. El recorrido medido con la cámara es de **210 mm**; el software limita los objetivos a 15–205 mm para no golpear los topes (era 25 mm; se redujo a 15 al tener el freno por interrupción, para cubrir más las esquinas de la portería).

- Desplazamiento en ambos sentidos con un puente H L298N.
- Velocidad de trabajo definida por calibración: portero izquierdo ≈ 490 mm/s con PWM 190; el derecho necesita PWM 255 por mayor fricción mecánica.
- A PWM 255 el carro izquierdo alcanzaba ~1 m/s y se trababa contra el final de carrera, por eso su velocidad se limitó.

5.3 Mecanismo de golpe

Un solenoide de empuje (push) de 12 V con un bloque de madera en el émbolo empuja la bola. Con pulsos de 80–120 ms el émbolo apenas se movía por la masa del bloque; con **350 ms** sale completo. El firmware impide activaciones repetidas: ignora una orden nueva hasta que pasaron el pulso más 500 ms de regreso del émbolo.

6. Diseño eléctrico y electrónico

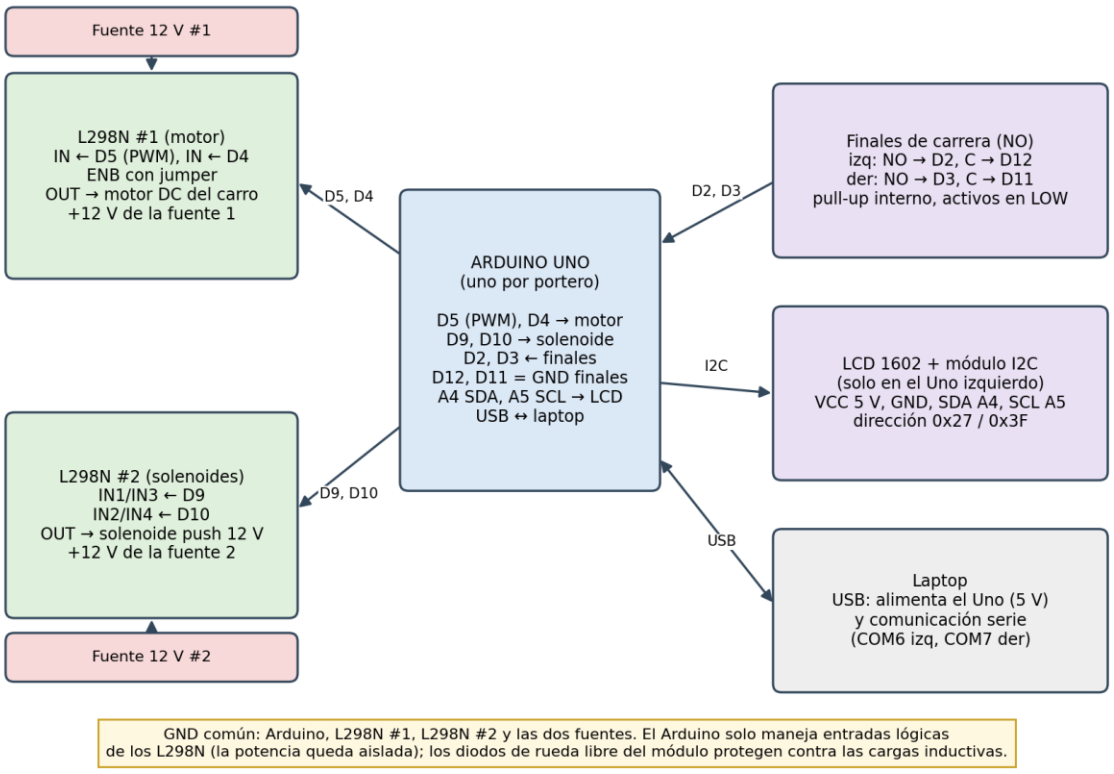


Figura 4. *Diagrama eléctrico de un portero (el segundo es idéntico, salvo la LCD).*

Tabla 7. *Asignación de pines del Arduino Uno.*

Pin	Función	Conectado a
D5 (PWM)	Velocidad y sentido del motor	L298N #1, entrada IN
D4	Sentido del motor	L298N #1, entrada IN
D9	Solenoide	L298N #2, IN1 e IN3
D10	Solenoide (nivel bajo)	L298N #2, IN2 e IN4
D2 (INT0)	Final de carrera izquierdo (NO)	Interruptor; su común va a D12
D3 (INT1)	Final de carrera derecho (NO)	Interruptor; su común va a D11
D12, D11	Salidas en LOW (tierra de los finales)	Común de cada final de carrera
A4, A5	I2C SDA, SCL	LCD 1602 con módulo I2C (solo Uno izquierdo)
USB	Alimentación 5 V y comunicación	Laptop (COM6 izquierdo, COM7 derecho)

6.1 Alimentación y protección

- Dos fuentes de 12 V independientes: una para los motores y otra para los solenoides, para que el pico de corriente del golpe no haga caer el voltaje del motor ni reinicie la lógica.
- Tierra común entre los Arduino, los dos L298N y las fuentes, necesaria para que las señales lógicas tengan la misma referencia.

- Aislamiento: el Arduino solo entrega señales lógicas a las entradas de los L298N; la corriente de potencia nunca pasa por el microcontrolador.
- Cargas inductivas: el módulo L298N incluye diodos de rueda libre que absorben el pico de voltaje al apagar el motor o el solenoide.
- Desconexión de potencia: apagando las fuentes de 12 V se detienen motores y solenoides sin afectar la comunicación del Arduino.
- Finales de carrera con pull-up interno y lectura confirmada (3 lecturas en LOW con 300 µs de separación) para filtrar el ruido del motor.

Se descartaron un módulo de relé (no conmutaba: estaba dañado) y el primer montaje de un solo L298N compartido entre motor y solenoide (el solenoide quedaba débil). Dos MOSFET IRFZ44N quedan como repuesto.

7. Arquitectura de software

7.1 Programas y archivos

Tabla 8. *Organización del código fuente.*

Archivo	Lenguaje	Responsabilidad
PORTERO_SERIAL/PORTERO_SERIAL.ino	C++ (Arduino + FreeRTOS)	Firmware de cada portero (mismo archivo en los dos Uno)
PORTERO_SERIAL_v1_loop/	C++ (Arduino)	Versión anterior sin RTOS, conservada como referencia
vision/vision_cancha.py	Python 3 + OpenCV	Programa principal: cámara, visión, decisiones, partida, registro
vision/trayectoria.py	Python 3	Filtro de Kalman, reflexión de rebotes, cruce, conversión a mm del carro
vision/calibracion.json	JSON	Esquinas de la homografía, resolución y color HSV
vision/analizar_registro.py	Python 3	Análisis de los CSV de cada partida
vision/medir_latencia.py	Python 3	Medición de la latencia de la cámara
vision/iniciar_cancha.bat	Batch	Instala dependencias y arranca el sistema con doble clic
vision/registros/	CSV + MP4	Evidencia de cada sesión: una fila y un cuadro de video por foto

Bibliotecas de terceros (sección 21 del enunciado): **FreeRTOS 11.1 para Arduino** (Phillip Stevens), **LiquidCrystal_I2C** y **Wire** en el firmware; **OpenCV**, **NumPy** y **pySerial** en la laptop. El código de control, percepción, predicción y reglas del juego es propio del equipo.

7.2 Ciclo principal de la laptop

Un hilo dedicado lee la cámara y conserva solo la foto más nueva, para no procesar imágenes atrasadas del búfer del controlador. El hilo principal, por cada foto: endereza la imagen, detecta la bola, actualiza el Kalman, calcula objetivos y golpes, envía los mensajes a los dos Arduino (como máximo cada 50 ms, y al menos cada 200 ms aunque no cambien, para alimentar el watchdog), aplica las reglas de gol y escribe el registro.

8. Implementación de tiempo real con FreeRTOS

El firmware usa la biblioteca FreeRTOS 11.1 para AVR. El tick del kernel lo genera el watchdog del ATmega328P cada **16 ms** (62,5 Hz); el planificador es expropiativo con prioridades fijas (0 a 3). Para no depender solo del tick, los eventos urgentes despiertan a la tarea de control con notificaciones directas.

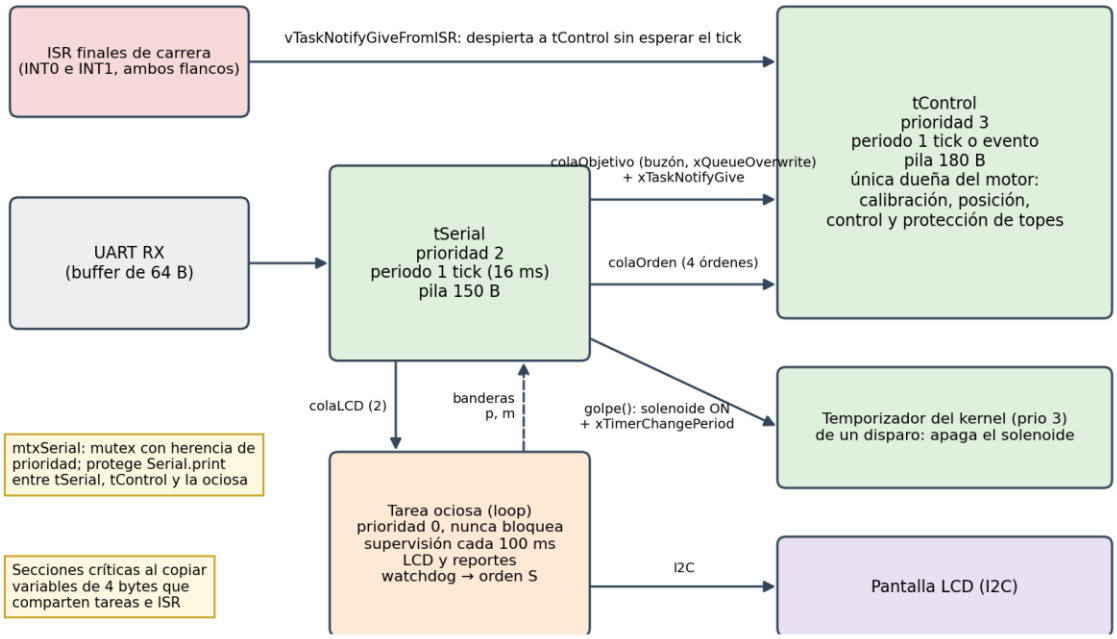


Figura 5. *Tareas, prioridades y mecanismos de comunicación del firmware.*

Tabla 9. *Tareas y componentes concurrentes (sección 9.1 del enunciado).*

Componente	Prioridad	Activación / periodo	Pila	Funciones del enunciado que cubre
ISR de finales (INT0, INT1)	Hardware	Cada flanco de un final de carrera	—	Adquisición de sensores (topes)
tControl	3 (máxima)	Cada tick (16 ms) o al instante con una notificación	180 B	Control de movimiento, protección de topes, calibración
Temporizador del kernel	3	Un disparo, al terminar el pulso	92 B (kernel)	Control del mecanismo de golpe (apagado seguro)
tSerial	2	Cada tick (16 ms)	150 B	Adquisición de datos de la visión, orden de golpe, puntos para la LCD
Tarea ociosa (loop)	0	Cuando no hay otra lista; supervisión cada 100 ms	192 B (kernel)	Supervisión y diagnóstico, pantalla, reportes
Laptop (Python)	—	Cada foto (≈ 15,6 ms)	—	Estimación de posición y trayectoria, máquina de estados del juego, goles

8.1 Justificación de prioridades

- **tControl tiene la prioridad más alta** porque es la única que puede dañar el hardware: si el carro llega a un extremo, cada milisegundo de retraso es empuje contra el tope. Además su trabajo es corto (decenas a cientos de microsegundos).
- **El temporizador del golpe también tiene prioridad 3** para que el solenoide se apague exactamente al terminar el pulso aunque otra tarea esté ocupada: el actuador nunca queda energizado de más.
- **tSerial tiene prioridad 2:** sus datos alimentan al control, pero un retraso de un tick (16 ms) es pequeño frente a la latencia de la cámara (≈ 100 ms).
- **La supervisión y la LCD corren en la tarea ociosa (prioridad 0):** la escritura I2C de la pantalla tarda varios milisegundos y nunca debe retrasar al control. Se ubicaron en el gancho de la tarea ociosa en lugar de una tarea propia porque el Uno tiene 2 KB de RAM: cada tarea adicional cuesta ≈ 200 B (pila + TCB de 48 B), y la tarea ociosa ya tiene su pila reservada. El código ahí nunca bloquea (colas y mutex con tiempo de espera 0).

8.2 Sincronización y comunicación

Tabla 10. *Mecanismos de sincronización usados.*

Mecanismo	Entre	Para qué
Notificación directa desde ISR (vTaskNotifyGiveFromISR)	ISR de finales → tControl	Despertar al control en cuanto cambia un final, sin esperar el tick; la ISR solo anota el instante
Buzón (cola de 1, xQueueOverwrite)	tSerial → tControl	Entregar siempre el objetivo más nuevo; uno viejo no sirve en tiempo real
Cola de órdenes (4 caracteres)	tSerial y supervisión → tControl	Órdenes manuales, recalibración y freno de seguridad S; tControl es la única que toca el motor
Cola de la LCD (2 mensajes)	tSerial y tControl → tarea ociosa	Desacoplar la escritura lenta por I2C
Mutex mtxSerial (con herencia de prioridad)	tSerial, tControl y tarea ociosa	Que dos tareas no mezclen texto en el puerto serie
Temporizador de un disparo (xTimerChangePeriod)	tSerial → servicio de temporizadores	Apagar el solenoide al cumplirse el pulso sin bloquear a nadie
Secciones críticas (taskENTER_CRITICAL)	Tareas e ISR	Copiar variables de 4 bytes de forma atómica en un procesador de 8 bits

8.3 Condiciones de carrera, bloqueos e inversión de prioridades

- **Un solo dueño del motor:** solo tControl escribe los pines del motor; las demás tareas le piden cosas por colas. Así no hay carreras sobre la dirección ni el PWM.
- **Variables de varios bytes:** `posMM`, `ultimoT` o los instantes de las mediciones ocupan 4 bytes y el AVR los escribe en varias instrucciones; se leen dentro de secciones críticas para no obtener valores a medias.
- **Inversión de prioridades:** si la tarea ociosa (prioridad 0) tiene el mutex del puerto serie y tControl (prioridad 3) lo necesita, el mutex de FreeRTOS hereda la prioridad y la ociosa termina rápido. Además, en modo automático tControl casi no imprime.

- **Bloqueos (deadlock):** solo existe un mutex y nunca se toma otro recurso mientras se tiene, por lo que no puede haber espera circular. La tarea ociosa nunca espera el mutex: si está ocupado lo intenta en la siguiente vuelta.
- **Pérdida de eventos:** el buzón de objetivos descarta a propósito los viejos; las órdenes manuales tienen cola de 4; las notificaciones de la ISR se acumulan como contador; el UART guarda 64 bytes, suficiente para 16 ms de mensajes (≈ 6 bytes por objetivo).
- **Sin esperas activas:** la calibración espera los finales de carrera durmiendo hasta la notificación de la ISR, las pausas usan vTaskDelay y el freno activo no espera: se pone y tControl lo suelta 40 ms después en un ciclo posterior, para que un objetivo nuevo no quede bloqueado.

8.4 Medición de tiempos de respuesta

El firmware mide sus propios tiempos con micros() y los informa con la tecla **m** (máximos desde la consulta anterior): periodo y tiempo de ejecución de tControl, tiempo de ejecución de tSerial, latencia final de carrera (ISR) → freno, latencia objetivo recibido → control, cantidad de golpes ejecutados e ignorados y la pila libre de cada tarea. La tabla siguiente compara la cota de diseño con lo medido en los dos Arduino el 5 de noviembre de 2026 (calibración + objetivos 40, 170 y 105 mm).

Tabla 11. *Tiempos de respuesta de los eventos críticos.*

Evento crítico	Cota por diseño	Medido (tecla m)
Final de carrera → motor cortado	< 1 ms (ISR + cambio de contexto + 3 lecturas de 300 µs)	0,76 ms (portero derecho)
Objetivo T recibido → control lo aplica	≤ 16 ms (lectura de tSerial) + < 1 ms (notificación)	84 µs desde que tSerial lo decodifica
Orden K → solenoide encendido	≤ 16 ms (lectura de tSerial)	≤ 16 ms; tSerial se ejecuta en ≤ 0,44 ms
Duración del pulso del golpe	350 ms ± 16 ms (resolución del tick)	22 ticks = 352 ms (configurado)
Periodo de tControl	1 tick = 16 ms	16,8 ms máx.; ejecución 0,15–0,30 ms
Pérdida de comunicación → freno	500 ms + ≤ 100 ms de supervisión	Comprobado (aviso SIN OBJETIVO y freno)
Cámara → decisión en la laptop	—	≈ 100 ms (medido, 2026-10-30)

8.5 Uso de memoria

Compilado para el Arduino Uno: 25 100 bytes de programa (77 %) y 730 bytes de variables globales (35 %), dejando 1 318 bytes para el montículo de FreeRTOS. Las pilas de las tareas, la tarea ociosa (192 B), el servicio de temporizadores (92 B + cola), las colas y el mutex ocupan ≈ 1 100 bytes. Para que alcanzara se pasaron los textos a la memoria flash (F(), PSTR) y se agruparon funciones de baja prioridad en la tarea ociosa. Medido con la tecla m: a tControl (180 B) le sobran 67–106 B y a tSerial (150 B) le sobran 54–87 B, por lo que ninguna pila corre riesgo de desbordarse.

9. Estados y operación

9.1 Estados del firmware de cada portero

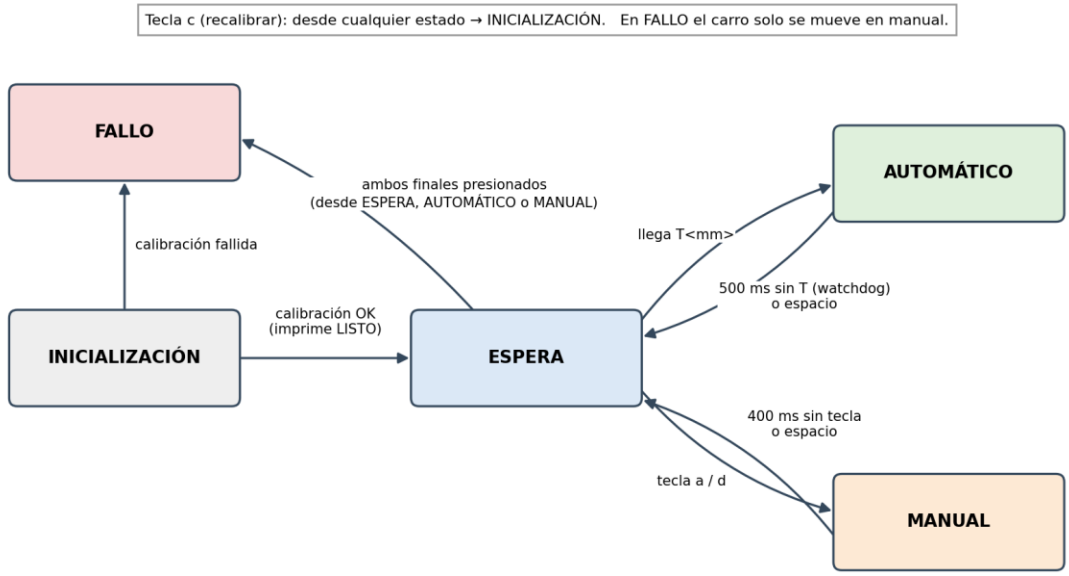


Figura 6. *Máquina de estados del firmware de cada portero.*

Tabla 12. *Estados del firmware.*

Estado	Qué hace	Sale cuando
INICIALIZACIÓN	Calibra: busca el tope izquierdo, mide el tiempo hasta el derecho, corrige el sentido del motor y el PWM, vuelve al centro	Calibración correcta (LISTO) → ESPERA; fallida → FALLO
ESPERA	Carro detenido y calibrado	Llega un objetivo → AUTOMÁTICO; tecla a/d → MANUAL
AUTOMÁTICO	Sigue los objetivos de la visión	500 ms sin objetivo o espacio → ESPERA
MANUAL	Movimiento con teclas para pruebas	400 ms sin tecla o espacio → ESPERA
FALLO	No acepta objetivos; solo movimiento manual	Tecla c (recalibrar) → INICIALIZACIÓN

9.2 Estados de la partida

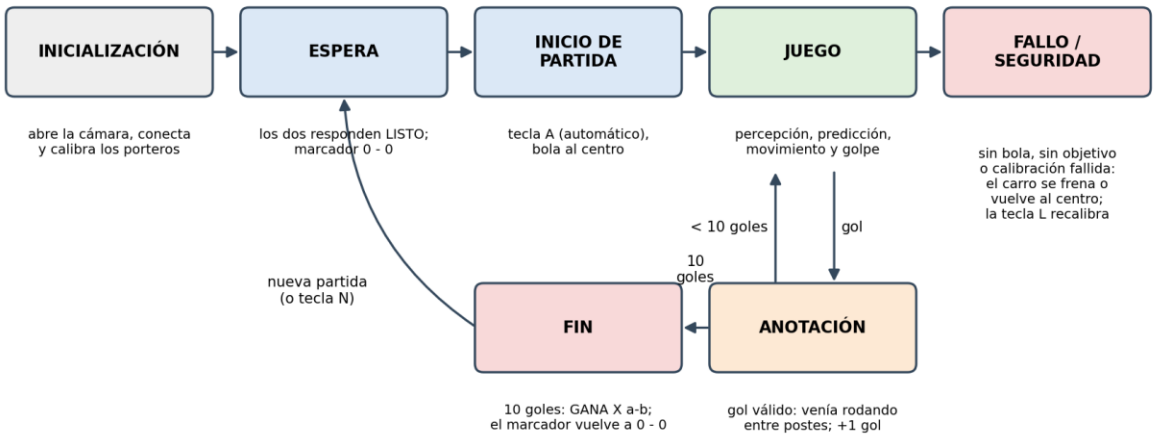


Figura 7. *Estados de la partida (sección 10 del enunciado), gestionados por la laptop.*

Tabla 13. *Correspondencia con los estados del enunciado.*

Estado del enunciado	Implementación
INICIALIZACIÓN	Se abre la cámara, se conectan los puertos y cada Arduino calibra su carro
ESPERA	Los dos porteros respondieron LISTO; el marcador está en 0-0
INICIO DE PARTIDA	Tecla A (automático) con la bola al centro; tecla N reinicia el marcador
JUEGO	Percepción, predicción, movimiento y golpe en cada foto
ANOTACIÓN	Se valida el gol, se suma al marcador, se actualiza la LCD y se bloquea 2 s
FIN	Un jugador llega a 10 goles: se anuncia «GANA X a-b» y el marcador vuelve a 0-0
FALLO / SEGURIDAD	Sin objetivo 500 ms → freno; sin bola 1 s → carros al centro; calibración fallida → FALLO

9.3 Operación

1. Encender las dos fuentes de 12 V y conectar los dos Arduino por USB.
2. Doble clic en `vision/iniciar_cancha.bat`. Cada portero se calibra solo y responde LISTO.
3. Si la cámara se movió: tecla K y clic en los 4 extremos de las paredes rectas.
4. Tecla N (marcador a 0-0), R (grabar evidencia) y A (modo automático).
5. Al terminar: Esc. Los registros quedan en `vision/registros`.

10. Percepción, predicción e intercepción

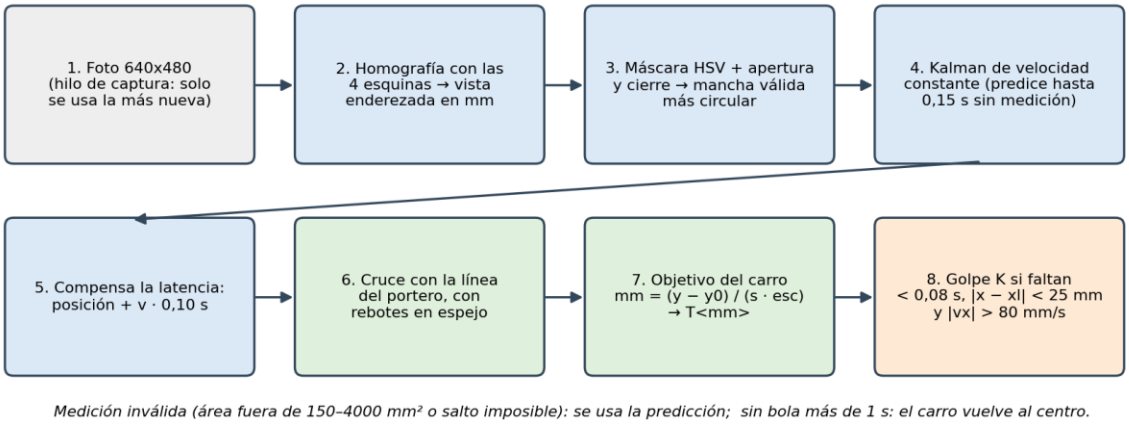


Figura 8. *Flujo de una foto, desde la cámara hasta las órdenes de movimiento y golpe.*

10.1 Percepción

La cámara (un iPhone usado como cámara web con iVCam) captura 640 x 480 a 60 fps. Con los 4 extremos de las paredes rectas marcados una sola vez se calcula una **homografía** que transforma la imagen en una vista cenital «enderezada» donde 1 mm = 1,5 píxeles; el origen está en la cara del portero izquierdo (x) y en la pared recta de arriba (y). La vista se extiende 40 mm fuera de la cancha para ver completa la bola pegada a la pared.

La bola se separa por color en el espacio HSV (H 95–130, S 100–255, V 50–255, ajustable en vivo). La máscara se limpia con apertura y cierre morfológicos y se elige la mancha de área válida (150 a 4 000 mm²) más circular. Su centroide es la medición (x, y) en milímetros.

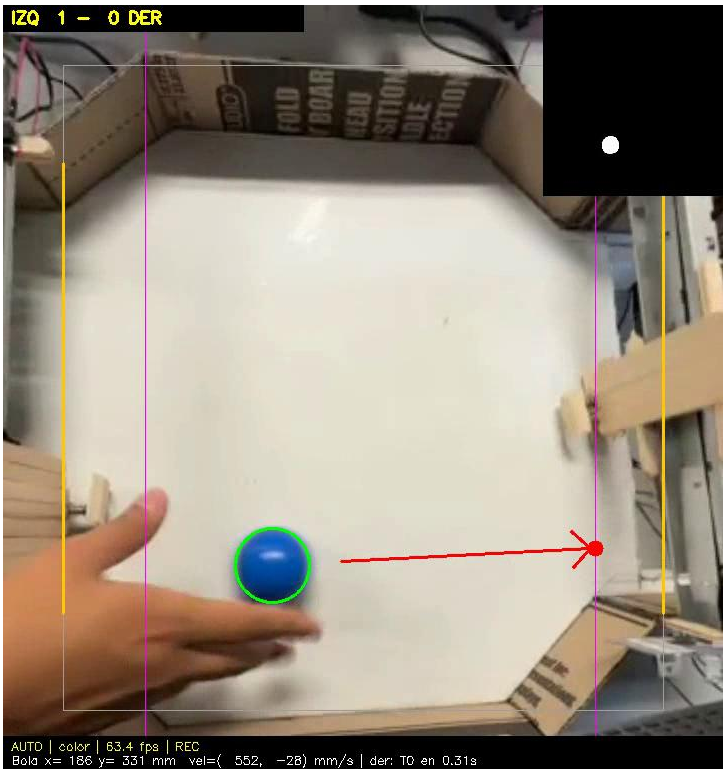


Figura 9. *Vista de trabajo: bola detectada (círculo verde), velocidad estimada y punto de intercepción predicho en la línea del portero derecho (punto rojo).*

10.2 Estimación (filtro de Kalman)

Se usa un filtro de Kalman de velocidad constante con estado [x, y, vx, vy] y medición [x, y]. El intervalo dt entre fotos se toma del reloj real. El ruido del proceso modela una aceleración aleatoria de 2 000 mm/s² (frenado y rebotes) y el de la medición 3 mm. El filtro suaviza la posición, estima la velocidad y, si la bola deja de verse, sigue prediciendo hasta 0,15 s; después se reinicia. La velocidad solo se usa a partir de la tercera medición.

10.3 Compensación de la latencia

Se midió una latencia de ≈ 100 ms entre lo que ocurre en la cancha y su procesamiento (cámara, transmisión iVCam y proceso), tanto por WiFi como por USB. Para decidir con la posición real se adelanta la estimación: $x \leftarrow x + vx \cdot 0{,}10$ y $y \leftarrow y + vy \cdot 0{,}10$.

10.4 Intercepción

Para cada portero, si la bola se acerca a su línea (x_l) a más de 50 mm/s, el tiempo de llegada es $t = (x_l - x)/vx$ y la altura de cruce sin paredes es $y^* = y + vy \cdot t$. Los rebotes en las paredes de arriba (y = 0) y abajo (y = 430 mm) se resuelven por reflexión en espejo: se toma y^* módulo 860 mm y, si pasa de 430, se refleja (860 − y^*). Si la bola se aleja, el portero sigue su altura; si no se ve durante 1 s, vuelve al centro.

La altura de cruce se convierte a la coordenada del carro con la recta medida para cada portero, $carro = (y^* - y_0)/(s \cdot e)$, donde y_0 es la altura del portero en el tope izquierdo, s = ±1 el sentido y e la escala. Se calibra moviendo cada carro a 5 posiciones y midiendo el bloque en la imagen (Figura 10).

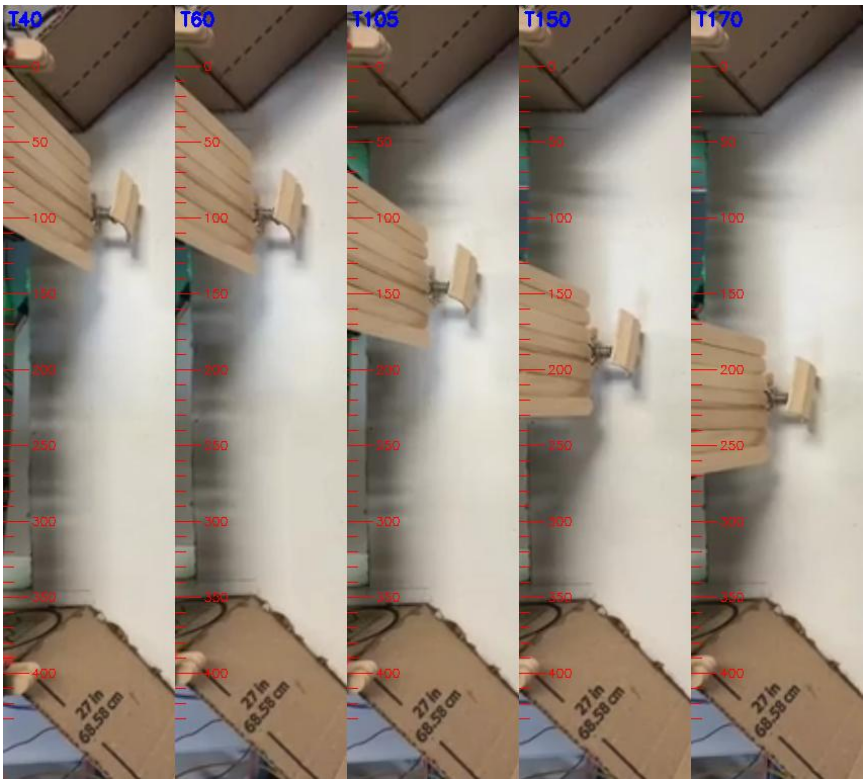


Figura 10. *Calibración de la posición del portero izquierdo: el carro en T = 40, 60, 105, 150 y 170 mm con la regla en mm de la vista.*

10.5 Decisión del golpe

El golpe se ordena cuando se cumplen a la vez: faltan menos de **0,08 s** para que la bola llegue a la línea, la bola está a menos de **25 mm** de la línea y viene a más de **80 mm/s**. Los tres umbrales se eligieron con una búsqueda en cuadrícula sobre las grabaciones del 3 de noviembre: con solo el criterio de tiempo el golpe salía 0,2–0,8 s antes porque la bola frena o rebota. Después de un golpe se esperan 0,9 s antes de otro (pulso de 350 ms más el regreso del émbolo).

10.6 Datos inválidos e incertidumbre

- Manchas demasiado pequeñas o grandes (reflejos, la mano) se descartan por área.
- Saltos imposibles entre fotos reinician la estimación.
- Sin medición se usa la predicción del Kalman hasta 0,15 s; luego se declara la bola perdida.
- Sin objetivos nuevos por 500 ms el Arduino frena solo (watchdog en el firmware).
- La laptop reenvía el objetivo cada 200 ms aunque no cambie, para que el watchdog no frene a un portero que ya está en su sitio.

10.7 Limitaciones del sensor

- Latencia de ≈ 100 ms: a 1 m/s la bola avanza 100 mm antes de procesarse; la compensación depende de que la velocidad no cambie en ese tiempo.
- La posición del carro no se mide: se estima por tiempo (error típico ±8 mm, hasta 14 mm).
- La cámara se recalibra cada vez que el teléfono se mueve.

- Con la mano o un objeto azul en la cancha pueden aparecer detecciones falsas.

11. Detección de anotaciones y marcador

11.1 Mecanismo

Primero se instaló un sensor infrarrojo en la portería (entrada A3 del Arduino), pero daba lecturas falsas por la luz ambiente y por la paleta del portero. Se reemplazó por la misma cámara, que ya conoce la posición de la bola respecto a la línea de gol. Con la regla inicial se contaron goles falsos (la bola quieta junto a la paleta o levantada con la mano); se endureció así:

- El centro de la bola debe pasar la cara del portero ($x < 0$ o $x > 400$ mm) entre los postes (y entre 65 y 365 mm, ±10 mm), o perderse de vista a menos de 15 mm de la cara durante 0,3 s (entró bajo el chasis).
- Y además debe venir rodando hacia esa portería a más de 150 mm/s en los últimos 0,5 s.

11.2 Validación y conteo único

- Tras un gol se bloquean nuevos goles durante 2 s.
- Y no se rearma la detección hasta que la bola vuelve a la zona central (x entre 100 y 300 mm). Así un mismo ingreso no se cuenta dos veces.

11.3 Marcador y ganador

Cada gol se registra en el estado de la partida, se imprime en consola, se muestra en la vista de la cámara y se envía a la LCD (línea 1: «IZQ: n DER: n»; línea 2: mensaje). Por decisión del equipo el marcador cuenta **goles anotados** en lugar de restar vidas: el primero que llega a 10 goles gana, se anuncia «GANA X a-b» y el marcador vuelve a 0-0. Es equivalente a la regla del enunciado (10 goles recibidos equivalen a 0 puntos de vida del rival) y resultó más claro para los espectadores.

12. Plan y resultados de pruebas

Antes de integrar se probaron por separado sensores, motores y actuadores (sección 13 del enunciado). Las sesiones de juego se grabaron con la tecla R: cada una deja un CSV con una fila por foto (tiempo, detección, posición medida y estimada, velocidad, objetivos y golpes) y un video MP4 anotado.

Tabla 14. *Plan y resultados de pruebas.*

#	Prueba	Método	Resultado
1	Finales de carrera	Presionar cada uno a mano y leer el estado (tecla p)	OK; se detectaron y corrigieron cables invertidos y en corto
2	Movimiento de cada nave	Teclas a/d y calibración automática	Izq.: 427 ms de recorrido = 492 mm/s a PWM 190. Der.: necesita PWM 255 (92–350 mm/s, fricción variable)
3	Solenoide	Tecla j (1 s encendido) y k (pulso)	Golpe completo con 350 ms; débil con 80–120 ms
4	Frecuencia de la cámara	Intervalo entre filas de los CSV (20 sesiones)	64 fps de mediana; intervalo p95 = 30 ms
5	Latencia de la cámara	medir_latencia.py (cambio de pantalla filmado)	≈ 100 ms por WiFi y 98 ms por USB
6	Calibración de la vista	Vista enderezada contra la regla en mm	Paredes en 0 y 430 mm; error < 5 mm
7	Posición del carro vs. visión	5 objetivos por portero, ajuste lineal	Izq. y = 35 + 1,08·c; der. y = 281 − 0,97·c; residuo típico ±8 mm, máx. 14 mm
8	Seguimiento de la bola	Grabación revisada cuadro por cuadro	Seguimiento continuo con la bola en juego, sin pérdidas en la prueba del 2026-11-02
9	Oportunidad del golpe	CSV con golpes vs. contacto en video (27 golpes)	≈ 40 % a tiempo (11 de 27) tras ajustar los umbrales
10	Detección de goles	Goles reales y casos trampa (bola quieta, mano)	Con la regla estricta se eliminaron los goles falsos (antes 4-4 falso)
11	Partida completa	Sesión de 455 s en automático	103 golpes registrados; partida sin intervención en el control
12	Tiempos de FreeRTOS	Tecla m tras calibrar y mover a 3 objetivos	Periodo 16,8 ms; final de carrera → freno 0,76 ms; objetivo → control 84 µs; pila libre ≥ 54 B

12.1 Validación del firmware con FreeRTOS

El firmware con FreeRTOS se cargó en los dos Arduino el 5 de noviembre de 2026. Los dos calibraron solos y respondieron LISTO, y siguieron los objetivos 40, 170 y 105 mm con error de 3 a 8 mm en el izquierdo y de 5 a 8 mm en el derecho. El golpe funcionó y repetirlo enseguida se ignoró como estaba previsto. Además se ejecutó mientras el carro calibraba, lo que demuestra que las tareas corren de forma concurrente. El portero derecho necesitó PWM 255 y llegó solo a 174 mm/s por la fricción de su riel, un problema mecánico y no del firmware. Falta repetir una partida completa (prueba 11) con esta versión. La versión anterior sin RTOS se conserva en `PORTERO_SERIAL_v1_loop` por si hace falta volver a ella durante la demostración.

13. Registro de problemas, decisiones y cambios de diseño

Tabla 15. *Problemas encontrados y cómo se resolvieron.*

Problema	Causa	Solución / decisión
La imagen se veía con zoom y deformada al calibrar	Esquinas marcadas en los postes de las porterías	Calibrar con los extremos de las paredes rectas y extender la vista 40 mm
Video con atraso	Latencia de cámara + iVCam (~100 ms)	Hilo de captura con la foto más nueva + compensación de latencia
El carro izquierdo se trababa en el tope	A PWM 255 iba a ~1 m/s	PWM 190 y objetivos limitados (hoy 15–205 mm)
Recorrido real distinto al supuesto	Se supuso 280 mm	Medido con la cámara: 210 mm
Posición inconsistente tras calibrar	Regreso al centro por tiempo fijo	Contar desde que se suelta el final de carrera (punto fijo)
Control proporcional no llegaba	A PWM bajo el motor casi no avanza	Control todo-o-nada a la velocidad calibrada
Motor del portero derecho al revés y lento	Cableado distinto y más fricción	Autocalibración: invierte el sentido y sube el PWM solo
«Lado contrario» falso al calibrar	El final de inicio seguía presionado	Contar el final contrario solo después de soltar el de inicio
Los dos finales leídos a la vez	Cables de D2/D3 en corto	Revisión de cableado + verificación en firmware (estado FALLO)
El solenoide no golpeaba	Jumper dañado, cable suelto, pin D8 sin señal, relé dañado	L298N dedicado en D9/D10; se cambió el jumper
Golpe débil	Un L298N compartido y pulsos cortos	L298N y fuente propios + pulso de 350 ms
El golpe paraba el motor	delay() bloqueaba todo el programa	Golpe no bloqueante; en FreeRTOS, temporizador del kernel
Golpe 0,2–0,8 s antes de tiempo	La bola frena o rebota	Criterio de tiempo + distancia + velocidad (búsqueda en cuadrícula)
Goles falsos (4-4)	Sensor IR y regla de gol simple	Gol por cámara con velocidad mínima, bloqueo y rearme
Detecciones falsas	Modo «fondo» activado por accidente	Usar el modo «color» (por defecto)
La vista se descalibra cada día	El teléfono se retira del laboratorio	Marcar la posición del teléfono; recalibrar con K y medir carros
Con FreeRTOS el Arduino no respondía al encender	delay() se redefine como vTaskDelay y se llamaba en setup(), antes de arrancar el planificador	Espera con millis() solo en setup(); en las tareas, vTaskDelay
El freno bloqueaba al control 48 ms	frenar() esperaba con vTaskDelay	Freno no bloqueante: tControl lo suelta en un ciclo posterior (objetivo → control pasó de 50 ms a 84 µs)
La LCD no mostraba el marcador	La pila de la tarea ociosa (192 B) se desbordaba con snprintf al escribir la pantalla (pila libre medida: 0 B)	Formateo manual sin snprintf y números sin el búfer de 33 B: quedan 47 B libres
Goles por las esquinas de la portería	El recorrido del carro no cubre los extremos de la portería	Objetivos hasta 15 mm del tope; se propone alargar el palito de la paleta hacia la esquina descubierta

Problema	Causa	Solución / decisión
RAM insuficiente para 5 tareas	2 KB de RAM del Uno	3 tareas + temporizador + supervisión en la tarea ociosa; textos en flash

14. Competencias adquiridas

- **Sistemas de tiempo real:** separar lo que tiene plazo duro (topes, pulso del solenoide) de lo que tolera atraso (pantalla, reportes); asignar prioridades por consecuencia del retraso; usar ISR, notificaciones, colas, mutex y temporizadores de FreeRTOS; razonar sobre carreras, bloqueos e inversión de prioridades.
- **Sistemas embebidos con recursos limitados:** presupuestar 2 KB de RAM entre pilas, colas y kernel; medir pilas libres; mover constantes a flash.
- **Visión por computadora:** homografía, segmentación por color, morfología, calibración de cámara y medición en milímetros.
- **Estimación y control:** filtro de Kalman, compensación de latencia, predicción con rebotes, control todo-o-nada con estimación por tiempo.
- **Electrónica de potencia:** puentes H, aislamiento entre lógica y potencia, cargas inductivas, fuentes separadas y tierra común.
- **Método de ingeniería:** probar cada componente por separado, registrar datos (CSV y video), decidir con mediciones (latencia, búsqueda en cuadrícula de los umbrales) y documentar los cambios.
- **Trabajo con hardware real:** diagnóstico de fallas intermitentes (cables, jumpers, relé dañado, fricción mecánica).

15. Evidencia audiovisual

Tabla 16. *Videos solicitados (sección 15 del enunciado).*

Video	Evidencia disponible	Estado
Construcción o montaje	Fotografías y videos del equipo durante el armado	Por recopilar
Pruebas de movimiento de cada nave	Calibración automática y modo manual	Por grabar con el firmware FreeRTOS
Prueba del golpe	Prueba con la tecla j/k	Por grabar
Calibración o validación de la detección	registros/20261030_*.mp4 y 20261102_*.mp4 (seguimiento con el estimador)	Disponible
Detección de anotaciones y marcador	registros/20261103_*.mp4 (marcador en pantalla y LCD)	Disponible
Integración hardware–software	registros/20261103_194123 (455 s, 103 golpes)	Disponible (CSV); video del sistema completo por grabar
Partida automática completa	Partida hasta 10 goles	Por grabar

Los videos de `vision/registros` muestran la información de diagnóstico que pide el enunciado: modo, cuadros por segundo, posición y velocidad de la bola, objetivo de cada portero, tiempo hasta el cruce y marcador.

16. Limitaciones y trabajo futuro

- **Golpe a tiempo ≈ 40 %:** el límite viene de la latencia de la cámara. Propuesta: mover el sensor infrarrojo a la paleta para que el Arduino dispare el golpe localmente con una interrupción, sin esperar a la laptop.
- **Posición sin encoder:** un encoder en el motor eliminaría el error por tiempo.
- **Portero derecho con fricción variable:** revisar la banda y el riel; su calibración falla algunos días.
- **Cobertura del portero izquierdo desplazada:** correr su riel ≈ 40 mm para centrarlo en la portería.
- **Recalibración de la cámara:** una base fija para el teléfono evitaría recalibrar cada día.
- **Paredes transparentes y bola de acero** como en la referencia, si se cambia la visión por color por otro método.

Anexo A. Comandos de operación y prueba

Tabla 17. *Teclas de la ventana de la cámara (laptop).*

Tecla	Acción
K	Recalibrar las esquinas de la cancha (4 clics)
M / G	Modo color/fondo; guardar el color de la bola
A	Modo automático on/off
R	Grabar registro CSV + MP4
N	Marcador a 0-0
L	Recalibrar los carros
C	Cambiar de cámara
Esc	Salir

Tabla 18. *Teclas del monitor serie (Arduino, 115200 baud).*

Tecla	Acción
a / d	Mover a la izquierda / derecha (mientras se mantiene)
x / espacio	Soltar / frenar
w / s	Subir / bajar el PWM manual
k	Golpe
j	Solenoide encendido 1 s (medir con multímetro)
c	Recalibrar
p / v	Estado / estado en vivo
m	Mediciones de tiempo real y pila libre

Anexo B. Fragmento del firmware: creación de las tareas

```
colaObjetivo = xQueueCreate(1, sizeof(int));        // buzón: siempre el objetivo más nuevo
colaOrden    = xQueueCreate(4, sizeof(char));
colaLCD      = xQueueCreate(2, sizeof(MsgLCD));
mtxSerial    = xSemaphoreCreateMutex();
tmrGolpe     = xTimerCreate("Golpe", 1, pdFALSE, NULL, finGolpe);   // de un disparo
xTaskCreate(tareaControl, "Ctrl", 180, NULL, 3, &hControl);
xTaskCreate(tareaSerial,  "Ser",  150, NULL, 2, &hSerial);
attachInterrupt(digitalPinToInterrupt(FIN_IZQ), isrTope, CHANGE);
attachInterrupt(digitalPinToInterrupt(FIN_DER), isrTope, CHANGE);

void isrTope() {                                    // solo marca el instante y despierta al control
  tIsrTope = micros();
  BaseType_t despertar = pdFALSE;
  vTaskNotifyGiveFromISR(hControl, &despertar);
  if (despertar == pdTRUE) portYIELD_FROM_ISR();
}
```